



6

Type-Checking and Validation with TypeScript

In this chapter, we'll explore the importance of **property validation** in React components for creating robust, bug-free applications. We'll introduce **TypeScript**, which is a powerful tool for static type-checking in JavaScript.

We'll guide you through setting up TypeScript in the project and cover its basic and advanced concepts. We'll also provide examples of how to use TypeScript for type-checking in React components.

By the end of this chapter, you'll have a solid foundation in property validation and type-checking, and be ready to create more predictable, reliable components using TypeScript.

The following topics will be covered in this chapter:

- Knowing what to expect
- An introduction to TypeScript
- Using TypeScript in React

Technical requirements

The code files for this chapter can be found on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter05>.

Knowing what to expect

In any application, predictability is key. A predictable application behaves in expected ways, reducing bugs, improving user experience, and simplifying maintenance. When we talk about predictability in the context of React, we often refer to how components behave based on the props they receive. Props is short for properties and serves as the inputs to React components, determining their behavior and rendering. This is where the concept of **props validation** comes into play.

The importance of props validation

Props validation is a way to ensure that components receive the right type of data. It's like a contract between components. When a component specifies the types of props it expects to receive, it makes a promise that it will behave in a certain way if it receives props of those types.

Props validation is crucial for a few reasons, as follows:

- **It helps catch errors early in the development process:** If a component receives a prop of an unexpected type, it might not behave as expected, leading to bugs that can be hard to track down. By validating props, we can catch these errors before they cause problems.
- **Props validation improves code readability:** By looking at a component's prop types, you can quickly understand what data the component expects to receive. This makes it easier to use and reuse components throughout your application.
- **Props validation makes components more predictable:** When a component clearly specifies the types of props it expects to receive, it's easier to understand how the component will behave based on its props.

Potential issues without props validation

Without adequate props validation, components can become unpredictable and prone to bugs. Let's take a look at a component:

```
const MyList = ({ list }) => (  
  <ul>  
    {list.map((user) => (  
      <li key={user.name}>  
        {user.name} ({user.email})  
      </li>  
    ))}  
  </ul>  
);
```

In this example, a component expects to receive a **list prop**, which should be an array of objects with name and email properties. If this component receives a list prop that is a string, a number, or even an array, but without objects, it might try to access `user.name` or `user.email`, which would result in an error.

These types of errors can be hard to debug, especially in larger applications with many components. It also might be difficult to understand what exactly we should provide to the component without reading every single line of code of this component. Errors can also lead to crashes or unexpected behavior in your application. But what if we can add a props validation to our components that can help us catch these errors early and ensure that your components behave as expected? Let's explore it.

Options for props validation

There are several tools you can use for props validation in React and React Native. One of these is `PropTypes`, a library that allows you to specify the types of props a component should receive. Another option is TypeScript, a statically typed superset of JavaScript that provides powerful tools for type-checking.

Now, I would like to show you examples of the `MyList` component with `PropTypes`. Take a look at this component:

```
import PropTypes from 'prop-types';
const MyList = ({ list }) => (
  <ul>
    {list.map((user) => (
      <li key={user.name}>
        {user.name} ({user.email})
      </li>
    ))}
  </ul>
);
MyList.propTypes = {
  list: PropTypes.arrayOf(
    PropTypes.shape({
      name: PropTypes.string.isRequired,
      email: PropTypes.string.isRequired,
    })
  ).isRequired,
};
```

In this example, we're using `PropTypes` to specify that the `list` prop should be an array of objects, and each object should have a `name` and an `email` property, both of which should be strings.

Next, let's take a look at the TypeScript example:

```
type User = {
  name: string;
  email: string;
};
```

```
type MyListProps = {  
  list: User[];  
};  
const MyList = ({ list }: MyListProps) => (  
  <ul>  
    {list.map((user) => (  
      <li key={user.name}>  
        {user.name} ({user.email})  
      </li>  
    ))}  
  </ul>  
);
```

In this TypeScript example, we're defining a `User` type and a `MyListProps` type. The `User` type is an object with a `name` and an `email` property, both of which are strings. The `MyListProps` type is an object with a `list` property, which is an array of `User` objects.

While both `PropTypes` and `TypeScript` offer valuable tools for props validation, we'll be focusing on `TypeScript` for the remainder of this book. `TypeScript` provides a more comprehensive and powerful approach to type-checking, and it's becoming increasingly popular in the React and React Native communities.

In the following chapters, all examples will use `TypeScript`. By the end of this book, you'll have a solid understanding of `TypeScript` and how to use it in your own React and React Native projects. So, let's dive in and start exploring the world of `TypeScript`!

Introduction to TypeScript

As we embark on this journey to learn about type-checking and validation, let's momentarily step away from React and React Native and turn our attention to TypeScript. You might be wondering, "What exactly is TypeScript?"

TypeScript is a statically typed superset of **JavaScript**, developed and maintained by Microsoft. This means that it adds additional features to JavaScript, one of the most significant being static typing. While JavaScript is dynamically typed, TypeScript introduces a type system that allows you to explicitly define the type of data that variables, function parameters, and function return values can have.

But don't worry, TypeScript is completely compatible with JavaScript. In fact, any valid JavaScript code is also valid TypeScript code. TypeScript uses a transpiler (a type of compiler) to convert TypeScript code, which browsers can't understand directly, into JavaScript code, which can run in any environment where JavaScript runs.

Consider the following JavaScript function:

```
function greet(name) {  
  return "Hello, " + name;  
}
```

```
console.log(greet("Mike")); // "Hello, Mike"  
console.log(greet(32)); // "Hello, 32"
```

This function works as expected when you pass a string as an argument. But if you pass a number, it doesn't throw an error, even though it doesn't make much sense to greet a number.

Now, let's see how we could write this function in TypeScript:

```
function greet(name: string) {  
  return "Hello, " + name;  
}  
console.log(greet("Mike")); // "Hello, Mike"  
console.log(greet(32)); // Error: Argument of type 'number' is not assignable to parameter of type 'string'.
```

In the TypeScript version, we've added a type annotation to the `name` parameter. This tells TypeScript that `name` should always be a string. If we try to call `greet` with a number, TypeScript will give us an error. This helps us catch the mistake before we even run the code.

This is a simple example, but it illustrates one of the key benefits of TypeScript: it can help us catch errors early before they lead to bugs in our code. It's like having a helpful co-pilot who points out potential issues before they become problematic.

Why use TypeScript?

Now that we've introduced what TypeScript is, let's delve into why you might want to learn and use it in your projects:

- **Catch errors early:** We've already discussed it, but it's worth putting it in the first place on the list. One of the biggest advantages of TypeScript is its ability to catch errors at compile time, before even running the code. This can help prevent many common errors that might not be caught until runtime in regular JavaScript.
- **Improve code readability:** TypeScript's **type annotations** make it clear what kind of values a function expects as arguments or what type of value a function returns. This can make the code easier to read and understand, especially for other developers who might be working on the same code base.
- **Easier refactoring:** TypeScript's static typing also makes it easier to refactor code. If you change the type of a variable or the signature of a function, TypeScript can help you find all the places in your code where you need to make corresponding changes.
- **Community and tooling support:** TypeScript has gained significant popularity in the JavaScript community and is used by many large companies like Microsoft, Google, and Airbnb. This implies that there's a large community of developers who can provide support and a wealth of resources for learning TypeScript. Additionally, many code editors have excellent support for TypeScript, providing features like **autocompletion**, **type inference**, and **error highlighting**.

- **Integration with modern frameworks and libraries:**

TypeScript integrates well with modern JavaScript frameworks like React and React Native, which have built-in TypeScript definitions, making it easier to build strongly typed applications.

Moreover, a vast majority of popular JavaScript libraries have TypeScript definitions available. These definitions, often contributed by the community, provide type information about the library's functions and objects, making it easier and safer to use these libraries in your TypeScript projects.

This widespread adoption of TypeScript in the JavaScript ecosystem ensures that you can leverage the benefits of TypeScript almost anywhere in your code base.

- **Increasing job market demand:** The popularity of TypeScript extends beyond just development practices: it's also increasingly sought after in the job market. Many companies, from start-ups to large corporations, are adopting TypeScript for their projects, and as a result, there's a growing demand for developers who are proficient in TypeScript. This is particularly true for roles involving React and React Native, where TypeScript is often used for its benefits in scaling and maintaining large code bases. By learning TypeScript, you're not only gaining a valuable skill for your projects but also making yourself more marketable as a developer.

In summary, TypeScript offers a range of benefits that can help you write more robust, maintainable code. It's a valuable tool in any

JavaScript developer's toolkit, and its growing popularity in the job market makes it a worthwhile investment for your career development.

But understanding the benefits of TypeScript is just the first step. To truly harness its power, you need to know how to use it in your projects. In the next section, we'll guide you through the process of setting up TypeScript in a React project. We'll cover everything from installing TypeScript to configuring your project to use it. So, let's dive in and start exploring the practical side of TypeScript!

Setting up TypeScript in a project

In the first chapter, we walked through the process of creating a new React project using **Vite**. Now, let's see how we can create a TypeScript project.

Vite provides a template for creating a new React and TypeScript project. You can create a new project with the following command:

```
npm create vite@latest my-react-app -- --template react-ts
```

This command creates a new Vite project with the `react-ts` template, which includes TypeScript. The project based on this template will include the `tsconfig.json` file in the root of your project. This file is used to configure TypeScript for your project.

Here's what the `tsconfig.json` file might look like:

```
{
  "compilerOptions": {
    "target": "esnext",
    "module": "esnext",
    "jsx": "react-jsx",
    "strict": true,
    "moduleResolution": "node",
    "esModuleInterop": true
  }
}
```

These settings tell TypeScript to compile your code to the latest version of JavaScript (`"target": "esnext"`), to use the latest module system (`"module": "esnext"`), and to use the new JSX transform introduced in *React 17* (`"jsx": "react-jsx"`). The `"strict": true` option enables a wide range of type-checking behavior to catch more issues.

With TypeScript set up, let's write some code. However, TypeScript uses different file extensions than JavaScript: `*.ts` for files without JSX, and `*.tsx` for files with JSX. So, let's create our first React component using TypeScript:

```
type AppProps = {
  message: string;
};
```

```
function App({ message }: AppProps) {  
  return <div>{message}</div>;  
}
```

In this example, we're defining an `AppProps` type for the props of the `App` component. This tells TypeScript that the `message` prop should be a string.

Now, let's take a look at the figure of how `main.tsx` looks like right now:

```
ReactDOM.createRoot(document.getElementById("root")!).render(  
  <React.StrictMode>  
    <App />   
  </React.StrictMode>  
);
```

Property 'message' is missing in type '{}' but required in type 'AppProps'.

Figure 6.1: App component in the main.tsx file with the error from TypeScript

This is how TypeScript checks and validates props usage in components. Here, we should pass the `message` prop:

```
ReactDOM.createRoot(document.getElementById("root")!).render(  
  <React.StrictMode>  
    <App message="Hello TypeScript" />  
  </React.StrictMode>  
);
```

Figure 6.2: App component in the main.tsx file without errors

Finally, you can run your project with the following command:

```
npm run dev
```

This command starts the **Vite development server**. If there are any **type errors** in your code, TypeScript will show them in the console as well.

Basic types in TypeScript

One of the key features of TypeScript is its rich type system. TypeScript introduces several basic types that you can use to describe the shape of your data. To specify the type of a variable, you use a colon after the variable name, followed by the type.

Let's explore these basic types:

- **Boolean:** The most basic datatype is the simple `true / false` value, which JavaScript and TypeScript call a Boolean:

```
let isDone: boolean = false;
```

- **Number:** As in JavaScript, all numbers in TypeScript are floating point values. These floating point numbers get the type number:

```
let age: number = 32;
```

- **String:** Another fundamental part of creating programs in JavaScript for web pages and servers alike is working with **textual data**. As in other languages, we use the type string to refer to these textual datatypes:

```
let color: string = "blue";
```

- **Array:** TypeScript, like JavaScript, allows you to work with arrays of values. Array types can be written in one of two ways. In the first, you use the type of the elements followed by `[]` to denote an array of that element type:

```
let list: number[] = [1, 2, 3];
```

The second way uses a generic array type, `Array<elemType>`:

```
let list: Array<number> = [1, 2, 3];
```

- **Tuple:** Tuple types allow you to express an array where the type of a fixed number of elements is known but does not need to be the same. For example, you may want to represent a value as a pair of a `string` and a `number`:

```
let x: [string, number];  
x = ["hello", 10]; // OK
```

- **Enum:** A helpful addition to the standard set of datatypes from JavaScript is the `enum`. As in languages like C#, an `enum` is a way of giving more friendly names to sets of numeric values:

```
enum Color {  
  Red,  
  Green,  
  Blue,  
}  
let c: Color = Color.Green;
```

- **Any:** We may need to describe the type of variables that we do not know when we are writing an application. These values may come from dynamic content, for example, from the user or a third-party library. In these cases, we want to opt out of type-checking and let the values pass through compile-time checks. To do so, we label these with the `any` type:

```
let notSure: any = 4;  
notSure = "maybe a string instead";  
notSure = false; // okay, definitely a Boolean
```

- **Unknown:** The `unknown` type is a type-safe counterpart of `any`. Anything is assignable to `unknown`, but `unknown` isn't assignable to anything but itself and `any` without a type assertion or a control flow-based narrowing. Likewise, no operations are permit-

ted on an `unknown` without first asserting or narrowing to a more specific type:

```
let notSure: unknown = 4;
notSure = "maybe a string instead";
// OK, because of structural typing
notSure = false;
let surelyNotAString: string = notSure; // Error, 'unknown' is not assignable to 'string'
```

In this example, we can't assign `notSure` to `surelyNotAString` without a type-check, because `notSure` is of the `unknown` type. This helps prevent errors because we can't inadvertently perform operations on variables of the `unknown` type without first checking their types.

A common use case for `unknown` is in a `catch` clause, where the type of the `error` object is not known:

```
try {
  // some operation that might throw
} catch (error: unknown) {
  if (error instanceof Error) {
    console.log(error.message);
  }
}
```

In this example, we don't know what the `error` type might be, so we give it the `unknown` type. This forces us to check its type before we can interact with it.

- **Void:** `void` is a little like the opposite of `any`: the absence of having any type at all. You may commonly see this as the return type of functions that do not return a value:

```
function warnUser(): void {  
    console.log("This is my warning message");  
}
```

- **Null and undefined:** In TypeScript, both `undefined` and `null` actually have their own types named `undefined` and `null` respectively. Much like `void`, they're not extremely useful on their own:

```
let u: undefined = undefined;  
let n: null = null;
```

However, `undefined` plays a crucial role in optional types. In TypeScript, you can make a type optional by adding `?` after the type name. This means the value can be of the specified type or `undefined`. For example:

```
function greet(name?: string) {  
    return 'Hello ${name}';  
}  
greet("Mike");  
greet(undefined); // OK  
greet(); // Also OK
```

- **Never:** The `never` type in TypeScript represents a type of value that never occurs. It's used in situations where a function never returns a value or reaches the end of its execution path. For example, a function that throws an error or one that has an infinite loop can be annotated with the `never` type:

```
function throwError(errorMsg: string): never {  
    throw new Error(errorMsg);  
}  
function infiniteLoop(): never {  
    while (true) {  
    }  
}
```

Understanding these basic types is a crucial first step in working with TypeScript. As you start to use TypeScript in your projects, you'll find that these types are powerful tools for writing robust, maintainable code.

In the next section, we'll delve deeper into TypeScript's type system and explore interfaces and type aliases, which provide a way to define complex types.

Interfaces and type aliases

While the basic types are useful for simple data types, when dealing with more complex data structures, we need more powerful tools.

This is where interfaces and type aliases come in. They allow us to define complex types and give them a name.

Interfaces

An interface in TypeScript is a way of defining a contract for complex types. It describes the shape an object should have. Here's an example:

```
interface User {  
  name: string;  
  email: string;  
}
```

In this example, we've defined a `User` interface with two properties: `name` and `email`, both of which are strings. We can use this interface to type-check objects:

```
const user: User = {  
  name: "Alice",  
  email: "alice@example.com",  
};
```

If we try to assign an object that doesn't match the `User` interface to the `user` variable, TypeScript will give us an error.

Type aliases

Type aliases are very similar to interfaces, but can be used for other types as well, not just objects. Here's an example of a `type` alias:

```
type Point = {  
  x: number;  
  y: number;  
};  
type ID = number | string;
```

In this example, we've defined a `Point` type that represents a point in a two-dimensional space and `ID` that can be a string or number. We can use these `type` aliases in the same way we use interfaces:

```
const point: Point = {  
  x: 10,  
  y: 20,  
};  
const id: ID = 100;
```

Interfaces vs type aliases

So, when should you use an interface, and when should you use a type alias? In many cases, the two are interchangeable, and it's mostly a matter of personal preference.

However, there are some differences. Interfaces are more extensible because they can be declared multiple times, and they will be merged together. Type aliases can't be re-opened to add new prop-

erties. On the other hand, type aliases can represent other types like union types, intersection types, tuples, and any other types that aren't currently available in an interface.

In general, if you're defining the shape of an object, either an interface or a type alias would work. If you're defining a type that could be something other than an object, you'll need to use a type alias.

In this section, we've taken our first steps into the world of TypeScript. We've learned about its setup in a **Vite** project, its basic types, and how to define complex types using interfaces and type aliases.

Now let's explore how we can use TypeScript with React components, state, event handlers.

Using TypeScript in React

Alright, we've made it this far! We've learned about the basics of TypeScript and talked about its benefits. Now, it's time to roll up our sleeves and get our hands dirty with some practical TypeScript in React.

In this section, we're going to explore how to use TypeScript to type-check all the different parts of a React application. We'll look at components, props, state, event handlers, context, and even refs. Don't worry: I'll walk you through plenty of examples to help illustrate these concepts.

Type-checking props in React components

In a React application, one of the primary areas where we can leverage TypeScript is in our components, specifically with props.

Let's see the example:

```
type GreetingProps = {  
  name: string;  
};  
const Greeting = ({ name }: GreetingProps) => {  
  return <h1>Hello, {name}!</h1>;  
};
```

In this example, we're defining a `GreetingProps` type that specifies the shape of the props that `Greeting` should receive. We're then using this type to type-check the `name` prop in the `Greeting` component.

This is a simple example with just one prop, but the same approach can be used for components with more complex props. For example, if a component receives an object or an array as a prop, we can define a type that describes the shape of that object or array. Here's an example:

```
type UserProps = {  
  user: {  
    name: string;  
    email: string;  
  };  
};
```

```
};  
};  
const UserCard = ({ user }: UserProps) => {  
  return (  
    <div>  
      <h1>{user.name}</h1>  
      <p>{user.email}</p>  
    </div>  
  );  
};
```

In this example, the `UserCard` component receives a user prop that is an object with name and email properties. We define a `UserProps` type that describes the shape of this object and use it to type-check the user prop.

Let's consider another common scenario in React: **optional props**. Sometimes, a component has props that aren't always required. In these cases, we can provide a default value for the prop, and mark it as optional in our type definition. Here's an example:

```
type ButtonProps = {  
  children: React.ReactNode;  
  disabled?: boolean;  
};  
const Button = ({ children, disabled = false }: ButtonProps) => {  
  return <button disabled={disabled}>{children}</button>;  
};
```


In the `ButtonProps` type, we're using `React.ReactNode` for the `children` prop. This is a special type provided by React that can accept any kind of renderable content. This includes strings, numbers, JSX elements, arrays of these types, or even functions that return these types. By using `React.ReactNode`, we're saying that the `children` prop can be any kind of content that React can render. Also, we're using the `disabled` prop, which is optional. We indicate that `disabled` is optional by adding a `?` after the prop name in the `ButtonProps` type. We also provide a default value of `false` for `disabled` in the component function parameters.

This way, we can use the `Button` component with or without the `disabled` prop, and TypeScript will still type-check it correctly:

```
<Button>Click me!</Button> // OK  
<Button disabled>Don't click me!</Button> // OK
```

Typing state

Just as we type-checked our props, we can also use TypeScript to type-check the state in our components. This ensures that we're always using the correct types of state values, providing another layer of safety to our code.

Let's look at an example of how we can apply TypeScript to state in a functional component:

```
const Counter = () => {  
  const [count, setCount] = React.useState<number>(0);  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <button  
        onClick={() => {  
          setCount(count + 1);  
        }}  
      >  
        Increment  
      </button>  
    </div>  
  );  
};
```

In this `Counter` component, we're using `React.useState<number>(0)` to declare a state variable `count` with an initial value of `0`. By providing `<number>` as a type argument to `useState`, we're telling TypeScript that `count` should always be a number. By the way: we can omit passing `<number>` because TypeScript is smart enough to infer that `count` should be a number based on the initial value type.

This also means that the `setCount` function will only accept numbers. If we try to call `setCount` with a non-number argument, TypeScript will give us an error.

Typing event handlers

Another area where TypeScript can be very useful in a React application is in **event handlers**. By type-checking our event handlers, we can ensure that we're using the correct event types and accessing the right properties on the event objects.

Let's look at an example of a functional component with an input field and a typed event handler:

```
const InputField = () => {  
  const [value, setValue] = React.useState("");  
  const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {  
    setValue(event.target.value);  
  };  
  return <input value={value} onChange={handleChange} />;  
};
```

In this `InputField` component, we're defining a `handleChange` function that will be called whenever the input field's value changes. We're using the `React.ChangeEvent<HTMLInputElement>` type for the event parameter to specify that this function should receive a change event from an input field.

This type includes all the properties we would expect from a change event on an input field, such as `event.target.value`. If we try to access a property that doesn't exist on this type of event, TypeScript will give us an error.

Typing context

When using TypeScript with React, we can also type-check our context to ensure that we're always using the correct types of values. Let's look at an example:

```
type ThemeContextType = {  
  theme: string;  
  setTheme: (theme: string) => void;  
};  
const ThemeContext = React.createContext<ThemeContextType | null>(null);  
const ThemeProvider = ({ children }: { children: React.ReactNode }) => {  
  const [theme, setTheme] = React.useState('light');  
  return (  
    <ThemeContext.Provider value={{ theme, setTheme }}>  
      {children}  
    </ThemeContext.Provider>  
  );  
};  
const useTheme = () => {  
  const context = React.useContext(ThemeContext);  
  if (context === null) {  
    throw new Error('useTheme must be used within a ThemeProvider');  
  }  
  return context;  
};
```

In this example, we're creating a `ThemeContext` with `React.createContext`. We're providing a `ThemeContextType` as a type argument to `createContext` to specify the shape of the con-

text value. This type includes a theme string and a `setTheme` function.

We're then creating a `ThemeProvider` component that provides the theme and `setTheme` values to the context. Inside the `useTheme` hook, we're using `React.useContext` to consume the `ThemeContext`. If the context is `null`, we throw an error.

This is a common pattern to ensure that the context is used within a provider.

With this example, I want to highlight an important TypeScript feature. In the `useTheme` hook, we don't specify the type. It returns the context value, which TypeScript knows is of the `ThemeContextType` type and not `null`, thanks to the error check. This means that when we use `useTheme`, TypeScript will automatically provide the correct, non-null context type.

Typing refs

Now, let's turn our attention to another powerful feature in React: refs. As you already know from *Chapter 3, Understanding React Components and Hooks*, refs give us a way to access **DOM nodes** or **React elements** directly within our components. But how do we ensure we're using refs correctly? TypeScript coming to the rescue.

Consider this example where we apply TypeScript to refs:

```
const InputWithRef = () => {  
  const inputRef = React.useRef<HTMLInputElement>(null);  
  const focusInput = () => {  
    if (inputRef.current) {  
      inputRef.current.focus();  
    }  
  };  
  return (  
    <div>  
      <input ref={inputRef} type="text" />  
      <button onClick={focusInput}>Focus the input</button>  
    </div>  
  );  
};
```

In this `InputField` component, we're creating a ref with `React.useRef`. We're providing `HTMLInputElement` as a type argument to `useRef` to specify the type of the ref. `HTMLInputElement` is a type provided by TypeScript's built-in DOM typings, and it represents an input element in the DOM. This type corresponds to the type of the DOM element that the ref is attached to.

This means that `inputRef.current` will be of the `HTMLInputElement | null` type, and TypeScript will know that it has a `focus` method.

Summary

In this chapter, we delved into the world of type-checking and validation in React. We started with the importance of props validation, then introduced TypeScript and its benefits for robust type-checking.

We then applied TypeScript to React, demonstrating its use in type-checking various aspects of React components, from props and state to event handlers, context, and refs. All of it allows you to create applications that are not only more dependable but also easier to maintain, allowing for early detection of errors and significantly improving both the quality of your code and your efficiency as a developer.

As we move to the next chapter, *Handling Navigation with Routes*, we'll shift our focus to navigation in React applications. We'll learn how to set up and use routes to navigate between different parts of our application.